

Mapping a database row to a Python object : Code-Along (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-v2-mapping-db-to-python-class>  <https://github.com/learn-co-curriculum/python-p3-v2-mapping-db-to-python-class/issues/new>

Learning Goals

- Map the values stored in a database row as attributes of a Python object.

Key Vocab

- **Object-Relational Mapping (ORM)**: a programming technique that provides a mapping between an object-oriented data model and a relational database model.

Introduction

The previous lesson showed how to implement methods to persist (i.e. save) the attributes of a Python object as a row in a database table.

In this lesson, we see how to map the opposite direction, namely we will map the values stored in a database table row to the attributes of a Python object. We will implement the following methods:

Method	Return	Description
instance_from_db (cls, row)	object that is an instance of <code>cls</code>	Return an object that is an instance of the class, assigning the attribute values from a table row

Method	Return	Description
<code>get_all(cls)</code>	a list containing objects that are instances of <code>cls</code>	Return a list of objects that are instances of the class, assigning the attribute values from each row in a table.
<code>find_by_id(cls, id)</code>	object that is an instance of <code>cls</code>	Return an object that is an instance of the class, assigning the attribute values using the table row specified by the primary key <code>id</code>
<code>find_by_name(cls, name)</code>	object that is an instance of <code>cls</code>	Return an object that is an instance of the class, assigning the attribute values using the first table row matching the specified <code>name</code>

Code Along

This lesson is a code-along, so fork and clone the repo.

NOTE: Remember to run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
pipenv install
pipenv shell
```

Let's continue building out the `Department` class and its object-relational mapping methods from the previous lesson. The starter code for the `Department` class is in `lib/department.py`.

We need to build a few methods to access one or more table rows and return Python objects with attribute values that match the row data.

To start, review the code from the `Department` class. Then take a look at this code in the `debug.py` file:

```
#!/usr/bin/env python3

from __init__ import CONN, CURSOR
from department import Department

import ipdb
```

```
def reset_database():
    Department.drop_table()
    Department.create_table()

    Department.create("Payroll", "Building A, 5th Floor")
    Department.create("Human Resources", "Building C, East Wing")
    Department.create("Accounting", "Building B, 1st Floor")

reset_database()
ipdb.set_trace()
```

This file is set up so that you can explore the database using the `Department` class from an `ipdb` session. The file recreates the `Department` table and persists a few objects into the table.

Run `debug.py` to confirm the code is able to reset and initialize the table:

```
python lib/debug.py
```

Then type the following statements (one at a time) at the `ipdb>` prompt to confirm the table rows:

```
ipdb> departments = CURSOR.execute('SELECT * FROM departments')
ipdb> [row for row in departments]
# => [(1, 'Payroll', 'Building A, 5th Floor'), (2, 'Human Resources', 'Building C, East Wing'), (3, 'Accounting', 'B
```

You can also use SQLITE EXPLORER to confirm the table content.

Maintaining a dictionary of objects persisted to the database

Several of the methods we write in this lesson will query the "departments" table and return a department object whose attributes are assigned to the row data. However, consider what happens if we query the table several times for the same table row. For example, we may have 10 employees working for the same department. Within our Python application, we will have 10 employee objects that should all be associated with the same department object. It is important that we avoid creating duplicate department objects when mapping from the same "departments" table row.

To solve this, we will cache (i.e. store) each department object that has been persisted to the database. We'll use a dictionary data structure, where each entry consists of a **key** and a **value**:

- the **key** is the `id` of the Python object that was saved to the database (i.e. an instance of the `Department` class).
- the **value** is the actual Python object that was saved to the database.

In later lessons we will use an ORM framework named **FLASK-SQLALCHEMY**, which manages the mapping between table rows and Python objects, and alleviates the need to explicitly maintain a dictionary in our code.

Let's update the `Department` class to add a class variable named `all` that references a dictionary to store each `Department` object that has been saved to the database.

```
from __init__ import CURSOR, CONN
```

```
class Department:
```

```
    # Dictionary of objects saved to the database.
```

```
    all = {}
```

```
    # existing methods ...
```

We'll also update the `save()` method to add the current `Department` instance to the dictionary, using the row's primary key as the dictionary key. This is done by adding the statement `type(self).all[self.id] = self` to the end of the method:

```
def save(self):
```

```
    """ Insert a new row with the name and location values of the current Department instance.
```

```
Update object id attribute using the primary key value of new row.
Save the object in local dictionary using table row's PK as dictionary key"""
sql = """
    INSERT INTO departments (name, location)
    VALUES (?, ?)
"""

CURSOR.execute(sql, (self.name, self.location))
CONN.commit()

self.id = CURSOR.lastrowid
type(self).all[self.id] = self
```

NOTE: `type(self)` will evaluate to the `Department` class.

Now we can implement the necessary methods for mapping table rows to Python objects.

instance_from_db()

Method	Return	Description
instance_from_db (cls, row)	object that is an instance of <code>cls</code>	Return an object that is an instance of the class, assigning the attribute values from a table row

This method will map the data stored in a "deparments" table row into an instance of `Department` .

One thing to know is that the database, SQLite in our case, will return a list of data for each row. For example, a row for the payroll department would look like this: `[1, "Payroll", "Building A, 5th Floor"]` . We use `row[0]` to get the department id `1` , `row[1]` to get the department name `"Payroll"` , etc.

Add the new class method `instance_from_db(cls, row)` to the `Department` class:

```

@classmethod
def instance_from_db(cls, row):
    """Return a Department object having the attribute values from the table row."""

    # Check the dictionary for an existing instance using the row's primary key
    department = cls.all.get(row[0])
    if department:
        # ensure attributes match row values in case local object was modified
        department.name = row[1]
        department.location = row[2]
    else:
        # not in dictionary, create new instance and add to dictionary
        department = cls(row[1], row[2])
        department.id = row[0]
        cls.all[department.id] = department
    return department

```

The `instance_from_db` method takes a reference to a Python class `cls` and a list named `row` that stores the column values from a table row. The method looks in the dictionary for an existing object using the primary key `id` as the dictionary key, and updates the attribute values to match the row data. However, it is possible that the database table may contain a row that does not correspond to a dictionary entry, in which case the method creates a new object and adds it to the dictionary.

Type `exit()` to terminate the debugging session, then rerun `python lib/debug.py` to create the table with initial values. We can list the dictionary of persisted objects:

```

ipdb> Department.all
{1: <Department 1: Payroll, Building A, 5th Floor>, 2: <Department 2: Human Resources, Building C, East Wing>, 3: <[

```

Execute a query to get a row of data, then use that row to get a Python Department object (type each statement one at a time at the prompt `ipdb>`):

```
ipdb> row = CURSOR.execute("select * from departments").fetchone()
ipdb> row
(1, 'Payroll', 'Building A, 5th Floor')
ipdb> department = Department.instance_from_db(row)
ipdb> department
<Department 1: Payroll, Building A, 5th Floor>
ipdb>
```

get_all()

Method	Return	Description
get_all (cls)	a list containing objects that are instances of <code>cls</code>	Return a list of objects that are instances of the class, assigning the attribute values from each row in a table.

To return all the departments in the database, we need to do the following:

1. define a SQL query statement to select all rows from the table
2. use the CURSOR to execute the query, and then call the `fetchall()` method on the query result to return the rows sequentially in a tuple.
3. iterate over each row and call `instance_from_db()` with each row in the query result to retrieve a Python object from the row data:

Add the new class method `get_all()` to the `Department` class:

```
@classmethod
def get_all(cls):
    """Return a list containing a Department object per row in the table"""
    sql = """
        SELECT *
        FROM departments
    """
```

```
rows = CURSOR.execute(sql).fetchall()

return [cls.instance_from_db(row) for row in rows]
```

With this method in place, let's try calling the `get_all()` method to access all the departments in the database.

Exit `ipdb` and run `python lib/debug.py` again and follow along in the `ipdb` session:

```
ipdb> Department.get_all()
[<Department 1: Payroll, Building A, 5th Floor>, <Department 2: Human Resources, Building C, East Wing>, <Department
ipdb>
```

Success! We can see all three departments in the database as a list of `Department` instances. We can interact with them just like any other Python objects:

```
ipdb> departments = Department.get_all()
ipdb> departments[0]
<Department 1: Payroll, Building A, 5th Floor>
ipdb> departments[0].name
'Payroll'
ipdb>
```

find_by_id()

Method	Return	Description
<code>find_by_id(cls, id)</code>	object that is an instance of <code>cls</code>	Return an object that is an instance of the class, assigning the attribute values using the table row specified by the primary key <code>id</code>

This one is similar to `get_all()`, with the small exception being that we have a `WHERE` clause to test the `id` in our SQL statement. To do this, we use a bound parameter (i.e. question mark) where we want the `id` parameter to be passed in, and we include `id` in a tuple as the second argument to the `execute()` method:

```

@classmethod
def find_by_id(cls, id):
    """Return a Department object corresponding to the table row matching the specified primary key"""
    sql = """
        SELECT *
        FROM departments
        WHERE id = ?
    """

    row = CURSOR.execute(sql, (id,)).fetchone()
    return cls.instance_from_db(row) if row else None

```

There are a couple important things to note here:

- Bound parameters must be passed to the `execute` statement as a sequence data type. This is typically performed with tuples to match the format that results are returned in. A tuple containing only one element must have a comma after that element, otherwise it is interpreted as a grouped statement (think **PEMDAS** → https://en.wikipedia.org/wiki/Order_of_operations).
- The `fetchone()` method returns the first element from `fetchall()`.

Let's try out this new method. Exit `ipdb` and run `python lib/debug.py` again:

```

ipdb> department = Department.find_by_id(1)
ipdb> department
<Department 1: Payroll, Building A, 5th Floor>
ipdb> department.name
'Payroll'
ipdb> department.location
'Building A, 5th Floor'
ipdb>

```

find_by_name()

Method	Return	Description
<code>find_by_name(cls, name)</code>	object that is an instance of <code>cls</code>	Return an object that is an instance of the class, assigning the attribute values using the first table row matching the specified <code>name</code>

The `find_by_name()` method is similar to `find_by_id()` , but we will limit the result to the first row matching the specified name.

```
@classmethod
def find_by_name(cls, name):
    """Return a Department object corresponding to first table row matching specified name"""
    sql = """
        SELECT *
        FROM departments
        WHERE name is ?
    """

    row = CURSOR.execute(sql, (name,)).fetchone()
    return cls.instance_from_db(row) if row else None
```

Let's try out this new method. Exit `ipdb` and run `python lib/debug.py` again:

```
ipdb> department = Department.find_by_name("Payroll")
ipdb> department
<Department 1: Payroll, Building A, 5th Floor>
```

Update the `delete` method to remove the dictionary entry

The `delete` method deletes the table row corresponding to the current Department instance. To ensure the Python object model reflects the table data, the `delete` method should also remove the corresponding key/value pair from the dictionary, and assign the instance `id` attribute back to `None` .

Update the `delete` method as shown:

```
def delete(self):
    """Delete the table row corresponding to the current Department instance,
    delete the dictionary entry, and reassign id attribute"""

    sql = """
        DELETE FROM departments
        WHERE id = ?
    """

    CURSOR.execute(sql, (self.id,))
    CONN.commit()

    # Delete the dictionary entry using id as the key
    del type(self).all[self.id]

    # Set the id to None
    self.id = None
```

Success!

Testing the ORM

The `testing` folder contains the file `department_orm_test.py`, which has been updated to test all of the ORM methods.

Run `pytest -x` to confirm your code passes the tests, then submit this code-along assignment using `git`.

Note: You may have to delete your existing database `company.db` for all of the tests to pass- SQLite sometimes "locks" databases that have been accessed by multiple modules.

Conclusion

In the previous lesson, we have created a database and persisted Python objects to the database. In this lesson, we saw how to map data stored in the database into Python objects. We cached Python objects using a dictionary to ensure a row maps to the same Python object each time it is retrieved using a query. Eventually, we will learn how to use an ORM framework that manages this mapping for us.

We haven't covered every use case, but we have a functioning ORM for a single Python class!

In the next lesson, we will see how to persist relationships between Python class instances.

Solution Code

```
#lib/department.py
```

```
from __init__ import CURSOR, CONN
```

```
class Department:
```

```
    # Dictionary of objects saved to the database.
    all = {}
```

```
    def __init__(self, name, location, id=None):
        self.id = id
        self.name = name
        self.location = location
```

```
    def __repr__(self):
        return f"<Department {self.id}: {self.name}, {self.location}>"
```

```
    @classmethod
```

```
def create_table(cls):
    """ Create a new table to persist the attributes of Department instances """
    sql = """
        CREATE TABLE IF NOT EXISTS departments (
            id INTEGER PRIMARY KEY,
            name TEXT,
            location TEXT)
        """
    CURSOR.execute(sql)
    CONN.commit()

    @classmethod
    def drop_table(cls):
        """ Drop the table that persists Department instances """
        sql = """
            DROP TABLE IF EXISTS departments;
        """
        CURSOR.execute(sql)
        CONN.commit()

    def save(self):
        """ Insert a new row with the name and location values of the current Department instance.
        Update object id attribute using the primary key value of new row.
        Save the object in local dictionary using table row's PK as dictionary key"""
        sql = """
            INSERT INTO departments (name, location)
            VALUES (?, ?)
        """
        CURSOR.execute(sql, (self.name, self.location))
        CONN.commit()

        self.id = CURSOR.lastrowid
```

```
type(self).all[self.id] = self
```

```
@classmethod
```

```
def create(cls, name, location):
```

```
    """ Initialize a new Department instance and save the object to the database """
```

```
    department = cls(name, location)
```

```
    department.save()
```

```
    return department
```

```
def update(self):
```

```
    """Update the table row corresponding to the current Department instance."""
```

```
    sql = """
```

```
        UPDATE departments
```

```
        SET name = ?, location = ?
```

```
        WHERE id = ?
```

```
    """
```

```
    CURSOR.execute(sql, (self.name, self.location, self.id))
```

```
    CONN.commit()
```

```
def delete(self):
```

```
    """Delete the table row corresponding to the current Department instance,  
    delete the dictionary entry, and reassign id attribute"""
```

```
    sql = """
```

```
        DELETE FROM departments
```

```
        WHERE id = ?
```

```
    """
```

```
    CURSOR.execute(sql, (self.id,))
```

```
    CONN.commit()
```

```
# Delete the dictionary entry using id as the key
```

```
del type(self).all[self.id]

# Set the id to None
self.id = None

@classmethod
def instance_from_db(cls, row):
    """Return a Department object having the attribute values from the table row."""

    # Check the dictionary for an existing instance using the row's primary key
    department = cls.all.get(row[0])
    if department:
        # ensure attributes match row values in case local object was modified
        department.name = row[1]
        department.location = row[2]
    else:
        # not in dictionary, create new instance and add to dictionary
        department = cls(row[1], row[2])
        department.id = row[0]
        cls.all[department.id] = department
    return department

@classmethod
def get_all(cls):
    """Return a list containing a Department object per row in the table"""
    sql = """
        SELECT *
        FROM departments
    """

    rows = CURSOR.execute(sql).fetchall()

    return [cls.instance_from_db(row) for row in rows]
```

```
@classmethod
def find_by_id(cls, id):
    """Return a Department object corresponding to the table row matching the specified primary key"""
    sql = """
        SELECT *
        FROM departments
        WHERE id = ?
    """

    row = CURSOR.execute(sql, (id,)).fetchone()
    return cls.instance_from_db(row) if row else None
```

```
@classmethod
def find_by_name(cls, name):
    """Return a Department object corresponding to first table row matching specified name"""
    sql = """
        SELECT *
        FROM departments
        WHERE name is ?
    """

    row = CURSOR.execute(sql, (name,)).fetchone()
    return cls.instance_from_db(row) if row else None
```

Resources

- [sqlite3 - DB-API 2.0 interface for SQLite databases - Python](https://docs.python.org/3/library/sqlite3.html)  [.\(https://docs.python.org/3/library/sqlite3.html\)](https://docs.python.org/3/library/sqlite3.html)

This tool needs to be loaded in a new browser window

Load Mapping a database row to a Python object : Code-Along (CodeGrade) in a new window